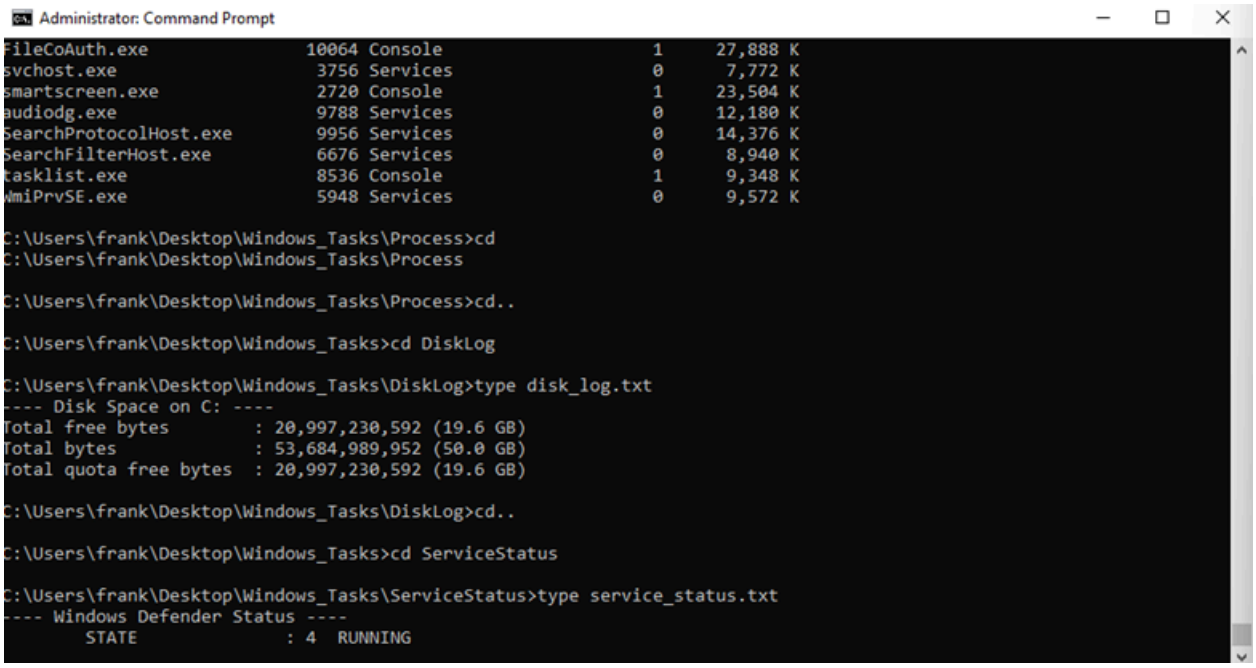
Frank Henry

Scripting Essentials

October 2nd, 2026

Windows CLI Batch Script



Above is a photo demonstrating a script written by me which automates specific tasks for review. The tasks that it shows are: The amount of disk space that all programs are using, the total amount of free and used bytes on the PC, and whether or not Windows Defender is Running, or Stopped.

The reason for this automation is for security concerns. It could be an issue if one program is taking up way more disk space than needed. An example of a security concern could be that if a hacker gets into your PC unknowingly, creates a file, and mines crypto with it, that file or program would be using so much disk space showing that outsiders have access to your PC. Another concern would be if Defender was STOPPED, it would allow hackers to get into your PC way more easily with no firewall/rules in place.

I wrote this script so that rather than manually checking everything every day, and wasting hours of time, 1 simple command that looks like “.\windows_tasks.bat” in the command

prompt runs the entire script within seconds for review. Now, that 19-character command is running this entire script simply:

“@echo off

cd /d "%~dp0"

echo ==== System Check Report ====

echo Date: %date% Time: %time%

echo.

echo ---- Running Processes ---- > Process\process.txt

tasklist >> Process\process.txt

type Process\process.txt

echo.

echo ---- Disk Space on C: ---- > DiskLog\disk_log.txt

fsutil volume diskfree C: >> DiskLog\disk_log.txt

type DiskLog\disk_log.txt

echo.

echo ---- Windows Defender Status ---- > ServiceStatus\service_status.txt

sc query WinDefend | findstr STATE >> ServiceStatus\service_status.txt

if %errorlevel%==0 (echo Windows Defender is running >> ServiceStatus\service_status.txt) else ( ) echo Windows Defender is NOT running >> ServiceStatus\service_status.txt type ServiceStatus\service_status.txt

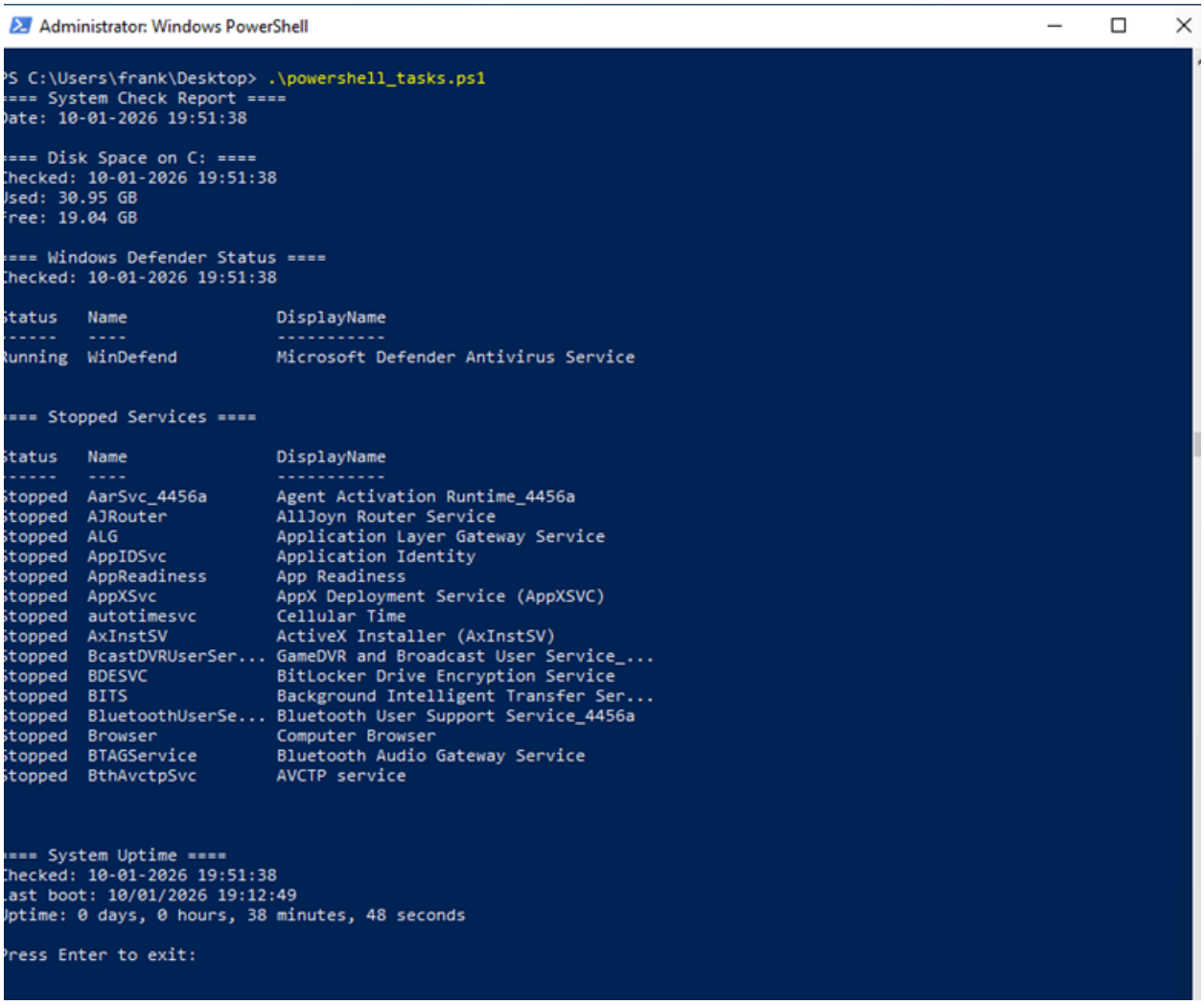
echo.

echo All tasks completed. Logs saved in personal directories!

pause”

So, as you can see, we simplified all that into a short command, and we get huge outputs from the Command Prompt to check for any security issues.

Powershell Script Output



Same thing here, but with different prompts because we did this in Powershell, and Powershell retrieves information a little differently. But for the most part, it's the same. Again, we retrieved total Disk Usage, program status' (we filtered for 'STOPPED' programs to make sure WinDefend was not in that list), and we checked how long the system has been running for, in other words, Uptime.

The command here is a simple ".\powershell_tasks.ps1". But it runs this entire script:

“

“# Disk Space, Service Status, and System uptime

===== System Check Script ======

Set-Location $PSScriptRoot

$time = Get-Date -Format "MM-dd-yyyy HH:mm:ss"

New-Item -ItemType Directory -Name Disk_Space -Force | Out-null

New-Item -ItemType Directory -Name Service_Status -Force | Out-null

New-Item -ItemType Directory -Name Uptime -Force | Out-null

"==== System Check Report ===="

"Date: $time"

""

Disk Space

$diskSpace = "Disk_Space\disk_space.txt"

$disk = Get-PSDrive C

usedGB=[math]::Round(disk.Used / 1GB, 2)

freeGB=[math]::Round(disk.Free / 1GB, 2)

"==== Disk Space on C: ====" | Out-File $diskSpace

"Checked: $time" | Out-File $diskSpace -Append

"Used: $usedGB GB" | Out-File $diskSpace -Append

"Free: $freeGB GB" | Out-File $diskSpace -Append

Get-Content $diskSpace

""

Service Status

$serviceStatus = "Service_Status\service_status.txt"

"==== Windows Defender Status ====" | Out-File $serviceStatus

"Checked: $time" | Out-File $serviceStatus -Append

Get-Service WinDefend | Out-File $serviceStatus -Append

"==== Stopped Services ====" | Out-File $serviceStatus -Append

Get-Service | Where-Object {$_.Status -eq "Stopped"} | Select-Object -First 15 | Out-File $serviceStatus -Append

get-Content $serviceStatus""

```
# System Uptime

$systemUptime = "Uptime\system_uptime.txt"

$boot = (Get-CimInstance Win32_OperatingSystem).LastBootUpTime

$uptime = (Get-Date) - $boot


"==== System Uptime ====" | Out-File $systemUptime

"Checked: $time" | Out-File $systemUptime -Append

"Last boot: $boot" | Out-File $systemUptime -Append

"Uptime: (uptime.Days) days, (uptime.Hours) hours, (uptime.Minutes) minutes,
(uptime.Seconds) seconds" | Out-File $systemUptime -Append

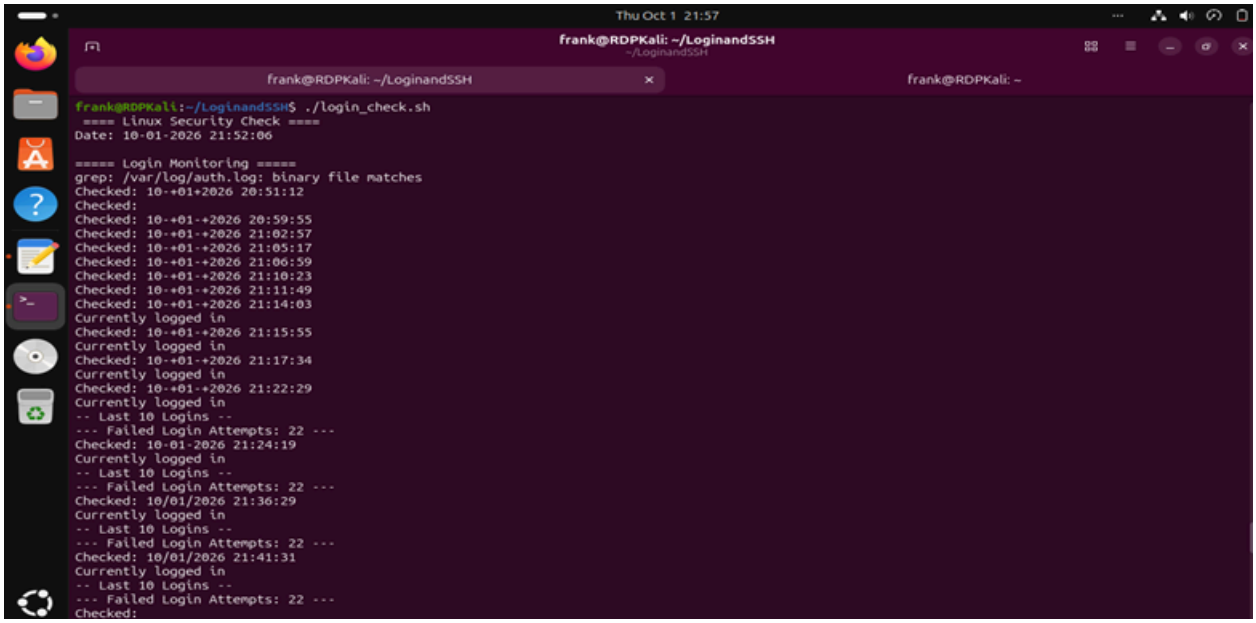

Get-Content $systemUptime

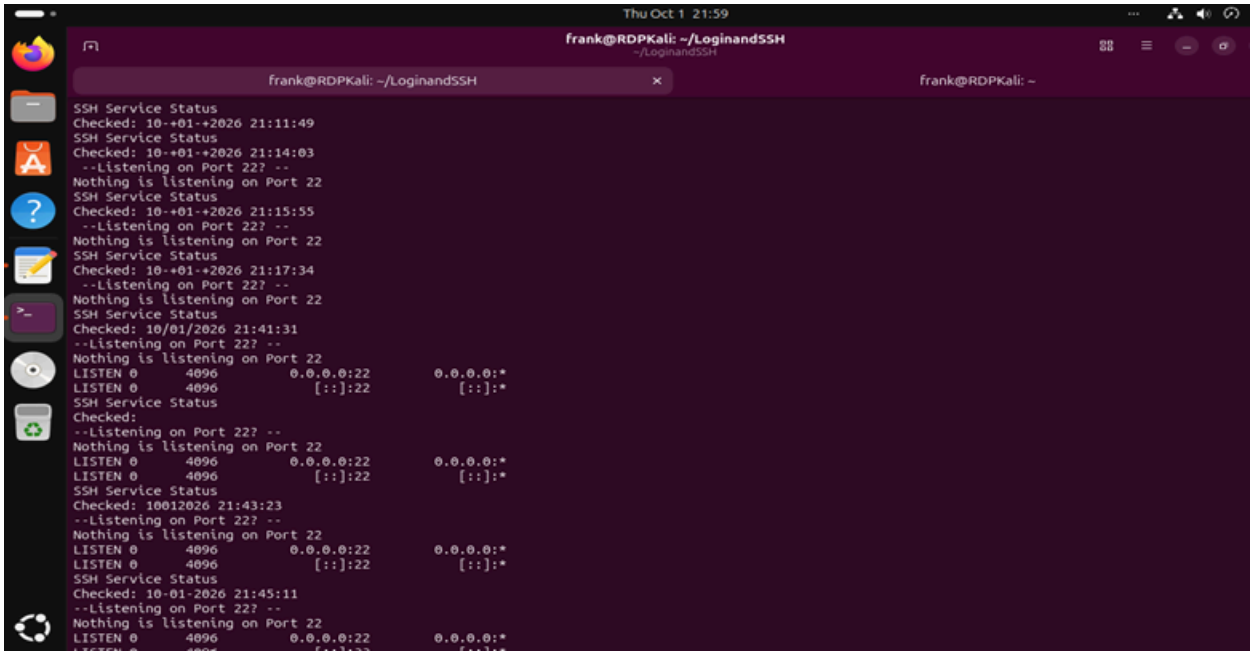
""


Read-Host "Press Enter to exit"

“
```

Bash (Linux) Script





These 2 screenshots above come from a Linux terminal. We created a script that outputs: a Login Monitoring system, and an SSH Service System. The command here would be

“./login_check.sh”, and if you noticed the “/” forward slash rather than the “\” backwards slash, that's because Linux uses slightly different command prompts.

The script from that command looks like this behind the scenes:

“

#!/bin/bash

Login and SSH Monitoring Script

cd "$(dirname "0")"

time=$(date "+%m-%d-%Y %H:%M:%S")

mkdir -p LoginLog SSHStatus

echo " ==== Linux Security Check ===="

echo "Date: $time"

echo

Login Monitoring

loginLog="LoginLog/login_log.txt"

echo "===== Login Monitoring ====="

```
echo "Checked: time">>"loginLog"


echo "Currently logged in" >> "loginLog"

who >> "loginLog"


echo "-- Last 10 Logins --" >> "loginLog"

last -n 10 >> "$loginLog"


failCount=$(grep -cE "Failed password|authentication failure" /var/log/auth.log)

echo "--- Failed Login Attempts: $failCount ---" >> "loginLog"

grep -E "Failed password|authentication failure" /var/log/auth.log | tail -n 5 >> "loginLog"


cat "loginLog"

echo


# SSH Service Status

sshLog="SSHStatus/ssh_status.txt"


echo "SSH Service Status" >> "sshLog"

echo "Checked: $time" >> "sshLog"
```

```
if systemctl is-active --quiet ssh; then
   echo "SSH: Running" >> "$sshLog"
elif systemctl is-active --quiet ssh.socket; then
   echo "SSH: Ready" >> "sshLog"
else
   echo "SSH: NOT RUNNING" >> "sshLog"
fi

echo "--Listening on Port 22? --" >> "sshLog"
ss -tln | grep ":22 " >> "sshLog" | echo "Nothing is listening on Port 22 " >> "sshLog"

cat "sshLog"

echo
echo "Scan Complete!"
“
```

Conclusion

As you can see, all the commands are running without error. But do not be fooled! We ran into some errors along the way. The one that I truly remember because it was the most frustrating and simple one was that I would forget to close quotes after writing a script line towards a folder! Meaning when I write out a “>> ‘loginLog’”, I would forget to close out the quotes so it would look like “loginLog.

Yes, I know, so simple and frustrating! But when we talk to computers, everything has to be perfect or they will not understand us.

I thoroughly enjoyed this project and chapter because it shows how we can truly simplify and automate computers, protect them and ourselves, and just be amazed by how far we’ve come as humanity. I learned that different programs use different commands, but they all pretty much do the same things. And I also learned how effective AI can be for security purposes, but AS WELL AS, how dangerous it can be for people trying to do malicious things online. I think the easiest CLI for me to use was the Windows command prompt, but I am excited to learn a lot more about Linux and Powershell as well!